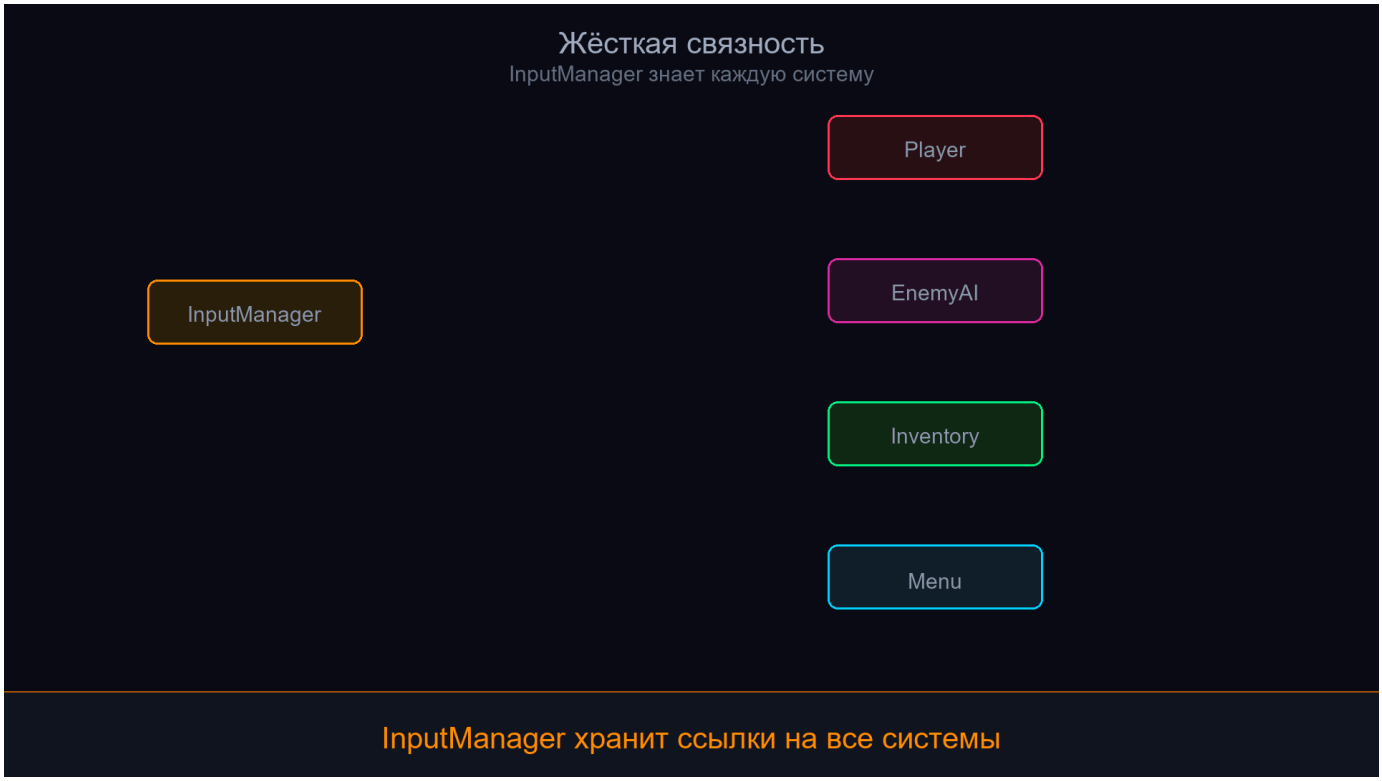
events

Событийная модель (Observer / Events)

Событийная модель (Observer / Events)



Проблема

В разделе про архитектуру мы разбирали пример: InputManager обрабатывает нажатие клавиши и каким-то образом передаёт эту информацию Player. Было сказано, что для этого используются события. Но как именно это работает?

Допустим, у нас есть InputManager, Player и EnemyAI. InputManager знает, какие клавиши были нажаты. Player должен двигаться при нажатии W, а EnemyAI должен перейти в состояние атаки при нажатии определённой клавиши (для дебага, к примеру).

Наивный подход: InputManager напрямую вызывает методы у Player и EnemyAI.

Python C#

```python
class InputManager:
    def __init__(self, player, enemy_ai, inventory, menu):
        self.player = player
        self.enemy_ai = enemy_ai
```

```
        self.inventory = inventory
        self.menu = menu

    def process_input(self, keys):
        if keys[pygame.K_w]:
            self.player.move_up()
        if keys[pygame.K_e]:
            self.inventory.open()
        if keys[pygame.K_ESCAPE]:
            self.menu.toggle()
        if keys[pygame.K_F1]:
            self.enemy_ai.debug_attack()
```

Что здесь не так? InputManager знает обо **всех** системах в игре. Он хранит ссылки на Player, EnemyAI, Inventory, Menu. Добавили новую систему - полезли в InputManager, добавили ещё одну ссылку и ещё один if. Убрали систему - снова лезем в InputManager. Он становится "узлом", через который проходит всё и который зависит от всего.

А если не только InputManager должен оповещать другие системы? Если Player при получении урона должен оповестить UI (обновить полоску здоровья), звуковую систему (проиграть звук) и систему частиц (показать кровь)? Тогда и Player тоже начнёт хранить ссылки на кучу систем. И каждая система будет знать обо всех остальных.

Это **сильная связанность** (tight coupling). Системы не независимы - они переплетены ссылками друг на друга. Изменение одной системы может сломать (сломает, не переживайте) другие.

Идея: событие как список

В архитектуре было сказано: "Упрощённо рассмотрим событие как список методов для вызова." Давайте разберём эту фразу.

Ну, идея такая: вместо того, чтобы InputManager напрямую вызывал `player.move_up()`, он говорит: "Была нажата клавиша W. Кому интересно - вот." И все, кому это интересно, реагируют самостоятельно.

Т.е. InputManager не знает, кто будет слушать. Он просто объявляет: "Произошло событие X." А другие системы заранее говорят: "Мне интересно событие X, вот метод, который нужно вызвать."

Это и есть **событийная модель** (она же паттерн **Observer**, она же **pub/sub** - publish/subscribe).

Три роли:

- **Издатель** (publisher): тот, кто объявляет о событии. Не знает, кто слушает.
- **Подписчик** (subscriber): тот, кто хочет реагировать на событие. Не знает, кто ещё подписан.
- **Событие** (event): механизм связи между ними. Хранит список подписчиков и вызывает их, когда издатель активирует событие.

Реализация

Простое событие

Ок. Самый базовый вариант: событие - это объект, который хранит список функций (от подписчиков) и может их все вызвать.

Python C#

```python
class Event:
    def __init__(self):
        self.listeners = []

    def subscribe(self, listener):
        self.listeners.append(listener)

    def unsubscribe(self, listener):
        self.listeners.remove(listener)

    def fire(self, *args):
        for listener in self.listeners:
            listener(*args)
```

Вот и всё. Три метода: подписаться, отписаться, активировать. Теперь посмотрим, как это меняет наш InputManager.

Применение: InputManager

Python C#

```python
class InputManager:
    def __init__(self):
        self.on_move_up = Event()
        self.on_move_down = Event()
        self.on_interact = Event()
        self.on_pause = Event()

    def process_input(self, keys):
        if keys[pygame.K_w]:
            self.on_move_up.fire()
        if keys[pygame.K_s]:
            self.on_move_down.fire()
        if keys[pygame.K_e]:
            self.on_interact.fire()
        if keys[pygame.K_ESCAPE]:
            self.on_pause.fire()

```

```python

# setup
input_manager = InputManager()

input_manager.on_move_up.subscribe(player.move_up)
input_manager.on_move_up.subscribe(camera_shake.on_player_move)
input_manager.on_interact.subscribe(inventory.try_open)
input_manager.on_pause.subscribe(menu.toggle)
```

Что изменилось? InputManager больше не знает ни о Player, ни о Menu, ни о чём другом. Он просто объявляет события. Кто на них подпишется - его не касается. Добавили новую систему? Подписали её на нужное событие. Убрали систему? Отписали. InputManager не трогаем.

Применение: урон и реакция на него

Рассмотрим ещё один пример. Игрок получает урон. На это должны отреагировать:

- UI: обновить полоску здоровья.
- Звук: проиграть звук боли.
- Камера: тряхнуть экран.
- Враги: возможно, агриться на источник урона.

Без событий Player должен знать обо всех этих системах. С событиями:

Python C#

```python
class Player:
    def __init__(self, health):
        self.health = health
        self.on_damage = Event()
        self.on_death = Event()

    def take_damage(self, amount):
        self.health -= amount
        self.on_damage.fire(self.health, amount)
        if self.health <= 0:
            self.on_death.fire()


# setup: each system subscribes to what it cares about
player.on_damage.subscribe(lambda hp, dmg: ui.update_health(hp))
player.on_damage.subscribe(lambda hp, dmg: sound.play("hit"))
player.on_damage.subscribe(lambda hp, dmg: camera.shake(intensity=dmg))

player.on_death.subscribe(lambda: ui.show_game_over())

```

```
player.on_death.subscribe(lambda: sound.play("death"))
player.on_death.subscribe(lambda: game.end_session())
```

Player знает только о своём здоровье и о том, что произошло событие. Кто отреагирует - да какая разница.

Встроенные события в pygame

Если вы работаете с pygame, то уже сталкивались с `pygame.event.get()` в игровом цикле и в конечных автоматах. Это встроенная очередь событий pygame - она собирает действия пользователя (нажатия клавиш, движение мыши, закрытие окна) и отдаёт их вам списком.

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE:
            # do something
            pass
```

Как это связано с нашим классом `Event`? `pygame.event` - это механизм получения **системных** событий от ОС и оборудования: клавиатура, мышь, окно. Наш `Event` - это механизм передачи **игровых** событий между системами: "игрок получил урон", "враг умер", "предмет подобран".

Т.е. они работают на разных уровнях. На практике это выглядит так: InputManager получает системные события через `pygame.event.get()`, интерпретирует их и публикует игровые события через наш `Event`:

```
class InputManager:
    def __init__(self):
        self.on_jump = Event()
        self.on_interact = Event()

    def process_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    self.on_jump.fire()
                elif event.key == pygame.K_e:
                    self.on_interact.fire()
        return True
```

Можно также создавать свои события в очереди pygame через `pygame.event.post()`, но для внутриигрового общения между системами наш подход с классом `Event` проще и понятнее. Да и лезть во внутренности чужой очереди не стоит.

> `pygame.event` отвечает за "что нажал пользователь". Наш `Event` отвечает за "что произошло в игре". Это разные задачи.

Встроенные события в C#

В C# есть встроенный механизм событий через ключевые слова `event` и `delegate`. Если вы работаете на C#, можно не писать свой класс `GameEvent`, а использовать стандартный синтаксис:

```
public class Player
{
    public int Health { get; private set; }

    // declare events with delegates
    public event Action<int, int> OnDamage;   // health, damage
    public event Action OnDeath;

    public Player(int health)
    {
        Health = health;
    }

    public void TakeDamage(int amount)
    {
        Health -= amount;
        OnDamage?.Invoke(Health, amount);
        if (Health <= 0)
            OnDeath?.Invoke();
    }
}

// subscribing
player.OnDamage += (health, damage) => ui.UpdateHealth(health);
player.OnDamage += (health, damage) => camera.Shake(damage);
player.OnDeath += () => ui.ShowGameOver();

// unsubscribing
player.OnDamage -= someMethod;
```

`?.Invoke()` проверяет, есть ли подписчики, и только тогда вызывает событие. Без этого, если подписчиков нет, будет `NullReferenceException`.

Оба варианта (свой GameEvent и встроенный event) работают. Встроенный - лаконичнее и идиоматичнее для C#. Свой класс - нагляднее и проще для понимания, что происходит "под капотом".

Событийная шина (Event Bus)

(aka диспетчер событий/сигналы-слоты/медиатор/центр оповещений и т.д.)

В примерах выше подписчик должен знать конкретного издателя: `player.on_damage.subscribe(...)` . Т.е. мы подписываемся на событие **конкретного** объекта. А что, если мы хотим реагировать на событие "любой враг умер"? Подписываться на каждого врага по отдельности?

Можно ввести **событийную шину** - центральный объект, через который проходят все события. Издатели отправляют события в шину, подписчики слушают шину.

Python C#

```python
class EventBus:
    def __init__(self):
        self.listeners = {}

    def subscribe(self, event_name, listener):
        if event_name not in self.listeners:
            self.listeners[event_name] = []
        self.listeners[event_name].append(listener)

    def fire(self, event_name, *args):
        for listener in self.listeners.get(event_name, []):
            listener(*args)


# global bus
bus = EventBus()

# enemy fires event when it dies
class Enemy:
    def die(self):
        bus.fire("enemy_died", self)

# score system listens
bus.subscribe("enemy_died", lambda enemy: score.add(100))
bus.subscribe("enemy_died", lambda enemy: sound.play("kill"))
```

Шина удобна, но у неё есть свои минусы: все события проходят через одну точку, типизация слабая (строковые имена событий легко опечатать), и при большом количестве событий становится сложно отслеживать, кто что слушает.

Типизированная шина

Хм, а можно ли избавиться от строковых имён? Можно. Вместо строки `"enemy_died"` используем **класс события** как ключ. Каждое событие - отдельный класс с нужными данными:

Python C#

```python
class EnemyDied:
    def __init__(self, enemy, position):
        self.enemy = enemy
        self.position = position

class PlayerDamaged:
    def __init__(self, health, amount):
        self.health = health
        self.amount = amount


class TypedEventBus:
    def __init__(self):
        self.listeners = {}

    def subscribe(self, event_type, listener):
        if event_type not in self.listeners:
            self.listeners[event_type] = []
        self.listeners[event_type].append(listener)

    def fire(self, event):
        for listener in self.listeners.get(type(event), []):
            listener(event)


bus = TypedEventBus()

# subscriber gets a typed object, not raw args
bus.subscribe(EnemyDied, lambda e: score.add(100))
bus.subscribe(EnemyDied, lambda e: particles.spawn_at(e.position))
bus.subscribe(PlayerDamaged, lambda e: ui.update_health(e.health))

# publisher creates an event object
bus.fire(EnemyDied(enemy=self, position=(self.x, self.y)))
```

Теперь опечатка в имени события - это ошибка компиляции (C#) или отсутствие класса (Python). Подписчик получает типизированный объект с конкретными полями вместо `object` или `*args`. А каждый класс-событие - ещё и документация: видно, какие данные несёт событие, без чтения кода издателя.

Когда использовать события?

События - не панацея. Используйте их там, где это оправдано:

- **Оповещение нескольких систем**: один факт (урон, смерть, подбор предмета) интересен многим. Это основной случай.
- **Ослабление связей**: если система A не должна знать о системе B, но должна как-то влиять на неё.
- **Расширяемость**: если вы хотите легко добавлять реакции на событие, не трогая издателя.

Не стоит использовать события для всего подряд. Если один объект вызывает метод другого объекта, и это всегда один конкретный вызов - обычный вызов метода проще и понятнее.

События усложняют отладку. Когда что-то пошло не так, сложнее отследить цепочку вызовов, потому что связь между издателем и подписчиком - неявная. Не используйте события без необходимости.

События в MVC, ECS и автоматах

Событийная модель хорошо ложится на MVC: Controller обрабатывает ввод и публикует события, Model слушает их и обновляет данные, View подписывается на изменения Model и перерисовывает интерфейс. Именно так работает "оповещение представления об изменениях", упомянутое в разделе про MVC.

В ECS события тоже полезны: System может публиковать событие при определённых условиях (столкновение, смерть), а другие System'ы - реагировать на них.

В конечных автоматах переходы между состояниями можно реализовать через события: вместо проверки клавиш внутри каждого состояния, состояние подписывается на нужные события от InputManager.

Т.е. событийная модель - это не отдельная архитектура, это **механизм связи** между частями любой архитектуры.

Что дальше?

Что	Где подробнее
Архитектура и разделение ответственности	Архитектура
Model-View-Controller и оповещение View	MVC

Что	Где подробнее
Конечные автоматы и переходы по событиям	Конечные автоматы
ECS и межсистемное общение	ECS
Тесная связность как типичная ошибка	Типичные ошибки

Powered by Wiki.js